

Reading Files

CSCI 1101

2024-03-05

Files and Paths

A **file** is a unit of persistently stored data on a computer.

Files are organized into a hierarchy of **directories**, often depicted as *folders*.

To specify a particular file or directory on your computer you can specify a **path** describing how to reach it, starting from either:

- your **current directory**, called “.”, (the default), or
- the **parent directory** of your current directory, called “..”, or
- the **root directory** on your computer, called “/”
(on Windows preceded by a drive letter like “C:”).

Make sure that you understand how to navigate the filesystem of your computer using the built-in file browser (e.g. *MacOS Finder* or *Windows Explorer*).

Paths in Python

To work with paths in Python, we need the “os” module:

```
import os
```

To get the current directory use the function:

```
os.getcwd    :    () --> str
```

To change the current directory use the function:

```
os.chdir    :    (str) --> NoneType
```

Paths in Python (ctd.)

To test whether a string represents a valid path on your computer use:

```
os.path.exists    :   (str) --> bool
```

To test whether a path specifies a file or a directory use:

```
os.path.isfile    :   (str) --> bool
```

```
os.path.isdir     :   (str) --> bool
```

To list the contents of a directory use:

```
os.listdir        :   (str) --> list str
```

Activity: Paths in Python

Expand the `sample_files.zip` file to the directory hierarchy `sample_files`.

Then use the python functions to:

- change to the `sample_files` directory,
- list its contents,
- determine whether `file_1.txt` is a file or a directory,
- determine whether `dir_1` is a file or a directory,
- list the contents of `dir_1`.

Reading Files

To read the contents of a file we must first **open** it with the `open` function.

The argument should be a string representing the path to a file.

The result is a **file** object, representing the file in python.

To access a file's contents use its `read` method.

After you are finished with the file you should **close** it with the `close` method.

```
def file_read(path) :  
    ''' sig: (str) --> str '''  
    file = open(path)  
    contents = file.read()  
    file.close()  
    return contents
```

Reading File Lines

You can read just a single line with the `readline` method:

```
def first_line(path) :  
    ''' sig: (str) --> str '''  
    file = open(path)  
    line = file.readline()  
    file.close()  
    return line
```

To read all remaining lines into a list of strings use the `readlines` method:

```
def all_lines(path) :  
    ''' sig: (str) --> list str '''  
    file = open(path)  
    lines = file.readlines()  
    file.close()  
    return lines
```

Aside: Named Function Arguments

If we try to print the lines of a file:

```
for line in all_lines("two_cities.txt") :  
    print(line)
```

Every other line comes out blank.

This is because each line in the file ends in a `\n` *and* the `print` function adds a `\n` as well.

We can tell `print` not to do this using a **named argument** called “end”:

```
for line in all_lines("two_cities.txt") :  
    print(line , end="")
```


Iterating Over Files

File objects are *iterable*, so you can read lines in a `for` loop:

```
def file_loop(path) :  
    contents = ""  
    file = open(path)  
    for line in file :  
        contents += line  
    file.close()  
    return contents
```

With Blocks

If we open a file but don't close it, either because we forget or because our program crashes, then our file object can get stranded.

To avoid this python has a construct that automatically closes an open file for us:

```
def file_with(path) :  
    with open(path) as file :  
        contents = file.read()  
        # no close necessary  
    return contents
```

It is good practice to open files using with blocks rather than opening and closing them manually.

Activity: Reading Files

Use a `with` block to write a function `every_other_line : (str) --> str` that takes a *path* to a file and returns the string consisting of every other line from that file.

For example:

```
> print(every_other_line("two_cities.txt") , end='')
```

```
It was the best of times,  
it was the age of wisdom,  
it was the epoch of belief,  
it was the season of Light,  
it was the spring of hope,  
we had everything before us,  
we were all going direct to Heaven,
```

To Do

Finish *Homework 6: More For-loops* on CodeRunner by 11:00AM on Thursday.

Finish on *Project 2 - What Does the Fox Say?* on CodeRunner by midnight Friday.